



Fine-tuning LLMs for Classification Tasks

Pirnavaskanthan Abeyakaran, Rishikesh Badari, Jatong Su

Georgia Institute of Technology, CS 4643/7643, Deep Learning

pabeyakaran3@gatech.edu

rishikesh.badari@gatech.edu

jsu77@gatech.edu

ABSTRACT

This project explores how well Large Language Models (LLMs) can classify handwritten digits from the MNIST dataset. Traditionally, Convolutional Neural Networks (CNNs) are used for this task, but we explore using LLMs with eight different image encoding methods: flat and row-wise variations of pixel-value sequences, normalized pixel-value sequences, base64 encoding, and binary inputs. Our goal is to find the best encoding method to adapt LLMs for image classification, showing their potential beyond just natural language processing.

INTRODUCTION

Large Language Models (LLMs) have achieved impressive results in various tasks. This project explores how well LLMs can adapt to image classification tasks usually handled by Convolutional Neural Networks (CNNs). Using the MNIST dataset, we test eight encoding strategies: flat and row-wise formats of pixel-value sequences, normalized sequences, base64, and binary inputs. By fine-tuning an LLM with each method, we aim to identify which representation best helps the model interpret images, potentially suggesting alternatives to CNNs.

PROBLEM-STATEMENT

Using the MNIST handwritten digit classification task as our benchmark, this project investigates which of our eight image encoding methods enables Llama-2-7b to best preserve spatial information when converting images to text for classification. We compare four encoding types (pixel-value sequences, normalized pixel-value sequences, base64, and binary), each implemented in both flat and row-wise formats, to determine how different representations and spatial preservation affect classification performance.

ENCODINGS

Pixel-Value Sequences:

1. **Flat:** Each 28x28 grayscale image is flattened into a single sequence of 784 pixel values (0-255).
1. : "0 0 0 ... 84 201 243 248 14 ... 0 0"
2. **Row-Wise:** The 28x28 image is preserved as 28 rows of 28 pixel values each.
1. "0 0 0 ... 84 201 243 248 ... 167 14 89 122 ... 0"

Normalized Pixel-Value Sequences:

1. **Flat:** Each 28x28 grayscale image is flattened into a single sequence of 784 normalized values (0-1) by dividing each pixel value by 255.
1. "0.0 0.0 0.0 ... 0.329 0.788 0.953 0.973 0.055 ... 0.0 0.0"
2. **Row-Wise:** The 28x28 image is preserved as 28 rows of 28 normalized values each.
1. "0.0 0.0 0.0 ... 0.329 0.788 0.953 0.973 ... 0.655 0.055 0.349 0.478 ... 0.0"

Base64 Sequences:

1. **Flat:** Images are converted to bytes and encoded in base64 format as a continuous string.
1. "iVBORw0KGgoAAAANSUh..."
2. **Row-Wise:** The base64 string is split into 28 segments corresponding to image rows.
1. "iVBORw0KGgoAAAA NSUhEgAAAAEAAAABCAAAAAA6"

Binary Sequences:

1. **Flat:** Images are converted to binary (0/1) by thresholding at 128, creating a sequence of 784 digits.
1. "0 0 0 ... 1 1 1 1 0 ... 0 0"
2. **Row-Wise:** The binary sequence is split into 28 rows of 28 digits each.
1. "0 0 0 ... 1 1 1 1 ... 0 0 0 0 ... 0"

RESULTS

Baseline Evaluation

The unmodified Llama-2-7b model with an added classification head:

Sequence Type	Flat Accuracy	Row-wise Accuracy
Pixel-Value	8.7%	11.0%
Normalized Pixel	11.0%	8.7%
Base64	9.6%	11.2%
Binary	11.1%	11.6%

Proposed Method Results

After fine-tuning with LoRA:

Sequence Type	Flat Accuracy	Row-wise Accuracy
Pixel-Value	98.7%	99.1%
Normalized Pixel	76.9%	68.0%
Base64	60.2%	95.9%
Binary	98.5%	98.8%

Pixel-Value Sequences

1. **Flat:** Only slightly higher than binary sequence performance despite more information in the input
2. **Row-Wise:** Highest accuracy most likely due to spatial and pixel information being preserved

Normalized Pixel-Value Sequences

1. **Flat:** Performed significantly worse than pixel-value and binary sequences
2. **Row-Wise:** Performed worse than flat version despite having more spatial information

Base64 Sequences

1. **Flat:** Showed limited improvement with compact representation, suggesting obscured spatial relationships
2. **Row-Wise:** Huge jump in accuracy once some spatial information is preserved

Binary Sequences

1. **Flat:** Achieved close to pixel-value accuracy, suggesting the positioning matters more than value
2. **Row-Wise:** Similar increase as pixel value sequences

CONCLUSIONS

LLM image classification testing revealed row-wise pixel value sequences performed best (99.1% accuracy), followed by row-wise binary sequences (98.8%), while base64 encoding underperformed (60.2%). Results indicate simple spatial representations enable LLMs to effectively classify images without CNN-like architectures.

Key Insights:

- Binary encoding's effectiveness shows spatial relationship preservation matters more than pixel value precision
- Base64's poor results demonstrate LLMs have difficulty with compressed, abstract image representations
- Similar performance between binary and pixel-value formats shows LLMs handle both simple and complex inputs well

Future Directions

- Study encoding performance on complex datasets beyond MNIST
- Research LLM-CNN hybrid architectures
- Improve classification via advanced prompt engineering without classification heads
- Study attention patterns to understand LLM spatial processing across encodings

Impact: LLMs can effectively process images with suitable encodings, extending beyond language tasks. Simple spatial representations may unlock further multimodal AI advances

REFERENCES

1. Arjun Singh, Nikhil Pandey, Anup Shirgaonkar, Pavan Manoj, and Vijay Aski. A study of optimizations for fine-tuning large language models. arXiv preprint arXiv:2406.02290, 2024
2. Xiaofei Sun, Xiaoya Li, Jiwei Li, Fei Wu, Shangwei Guo, Tianwei Zhang, and Guoyin Wang. Text classification via large language models. arXiv preprint arXiv:2305.08377, 2023. 2
3. Zhaoheng Zheng, Jingmin Wei, Xuefeng Hu, Haidong Zhu, and Ram Nevatia. Large language models are good prompt learners for low-shot image classification. arXiv preprint arXiv:2312.04076, 2023. CVPR 2024.